

Operating System Lab Assignment - 5



**SCHOOL OF ENGINEERING & TECHNOLOGY
K. R. MANGALAM UNIVERSITY, GURUGRAM,
HARYANA, INDIA**

Submitted to:

Atisha Ma'am

Subject Code:

ENCS351

Submitted by:

Garvit Pant

2401730088

B.Tech CSE (AI & ML)

Lab1 > Q4_Metrics_Threads_IPC > src > Program4.py

```
1  from threading import Thread, current_thread
2  from multiprocessing import Process, Pipe, Value
3
4  processes = [
5      {"pid": "P1", "arrival": 0, "burst": 7},
6      {"pid": "P2", "arrival": 2, "burst": 4},
7      {"pid": "P3", "arrival": 4, "burst": 1},
8      {"pid": "P4", "arrival": 5, "burst": 4},
9  ]
10
11  intervals = [
12      ("P1", 0, 2), ("P2", 2, 4), ("P3", 4, 6),
13      ("P3", 6, 7), ("P2", 7, 9), ("P4", 9, 11),
14      ("P1", 11, 13), ("P2", 13, 15), ("P1", 15, 16)
15  ]
16
17  def calculate_metrics():
18      print("\n--- SCHEDULING METRICS ---")
19      print("PID AT BT CT TAT WT RT")
20
21      tat_total = wt_total = rt_total = 0
22
23      for p in processes:
24          runs = [x for x in intervals if x[0] == p["pid"]]
25          first_start = runs[0][1]
26          completion = runs[-1][2]
27
28          tat = completion - p["arrival"]
29          wt = tat - p["burst"]
30          rt = first_start - p["arrival"]
```

Lab1 > Q4_Metrics_Threads_IPC > src > Program4.py

```
17  def calculate_metrics():
23      for p in processes:
31
32          tat_total += tat
33          wt_total += wt
34          rt_total += rt
35
36          print(p["pid"], p["arrival"], p["burst"],
37                completion, tat, wt, rt)
38
39      n = len(processes)
40      print("Average TAT=", round(tat_total/n, 2))
41      print("Average WT=", round(wt_total/n, 2))
42      print("Average RT=", round(rt_total/n, 2))
43
44  def show_gantt_chart():
45      print("\n--- ROUND ROBIN GANTT CHART ---")
46      print(" | ".join(pid for pid, _, _ in intervals))
47      times = [intervals[0][1]] + [end for _, _, end in intervals]
48      print(" ".join(str(t) for t in times))
49
50  def thread_task(name):
51      print(name, "running | Thread ID:", current_thread().ident)
52
53  def thread_demo():
54      print("\n--- THREADS DEMO ---")
55
56      t1 = Thread(target=thread_task, args=("Thread 1",))
57      t2 = Thread(target=thread_task, args=("Thread 2",))
58
```

Lab1 > Q4_Metrics_Threads_IPC > src > Program4.py

```
53 def thread_demo():
54
55     t1.start()
56     t2.start()
57
58     t1.join()
59     t2.join()
60
61     print("Both Threads Completed.")
62
63
64
65
66
67
68 def child_pipe(conn):
69     conn.send("Hello Parent - message from Child")
70     conn.close()
71
72
73 def pipe_demo():
74     print("\n-- PIPE IPC DEMO ---")
75
76     parent_conn, child_conn = Pipe()
77     child = Process(target=child_pipe, args=(child_conn,))
78
79     child.start()
80     message = parent_conn.recv()
81     child.join()
82
83     print("Parent received:", message)
84
85
86 #Shared Memory Demonstration
87 def update_shared(value):
```

Lab1 > Q4_Metrics_Threads_IPC > src > Program4.py

```
53 def thread_demo():
54
55     t1.start()
56     t2.start()
57
58     t1.join()
59     t2.join()
60
61     print("Both Threads Completed.")
62
63
64
65
66
67
68 def child_pipe(conn):
69     conn.send("Hello Parent - message from Child")
70     conn.close()
71
72
73 def pipe_demo():
74     print("\n-- PIPE IPC DEMO ---")
75
76     parent_conn, child_conn = Pipe()
77     child = Process(target=child_pipe, args=(child_conn,))
78
79     child.start()
80     message = parent_conn.recv()
81     child.join()
82
83     print("Parent received:", message)
84
85
86 #Shared Memory Demonstration
87 def update_shared(value):
```